Q A L A

Platform Concept & Architecture

Solution Factories · Environments · Models · Solutions

Document Type	Platform Concept, Architecture & Reference Specification
Version	v1.0
Scope	Full platform: root factory, solution factories, SDEs, solution models, solutions, playbooks, solution books, toolchains, repositories, and all subsystems
Audience	Product, Engineering, Design, Founders, Enterprise Architects, Onboarding Teams
Core Principle	Every problem-solving effort — personal project or global enterprise — is a solution. Qala governs them all.

Contents

1 Introduction

Qala is a universal platform for the creation, governance, development, and lifecycle management of solutions. Whether you are a solo developer organizing a personal project, a startup shipping a product, or a global enterprise managing thousands of services — Qala provides a single standardized interface through which all solution activity flows.

1.1 What is a Solution?

A solution is any purposeful output designed to address a problem, fulfill a goal, achieve an objective, or produce an intended outcome. Solutions span every domain and scale:

Domain	Example Solutions	Scale
Personal / Hobby	A custom home automation setup, a personal finance tracker, a self-hosted media server, a personal knowledge base.	Single user
Small Team / Startup	A SaaS product, a mobile app, an API service, an internal operations tool, a design system.	2–50 users
Professional Services	A consulting methodology, a managed IT service, a legal product, a tax compliance workflow, an advisory framework.	1–500 users
Enterprise	An ERP implementation, a pharmaceutical formulation, a financial model, a supply chain protocol, a regulatory system.	500–100,000+ users
Platform / Ecosystem	A software platform, an industry standard, an open-source framework, a developer SDK, a marketplace.	Millions of users

The unifying characteristic of all solutions is structure: they have identity, purpose, versions, changes, states, artifacts, documentation, and lifecycle. Qala governs that structure universally.

1.2 The Qala Philosophy

The Root Principle

Qala is itself a solution factory — the root solution factory from which all other factories, environments, models, and solutions descend.

Every entity in Qala is a solution or produces solutions. Every user is operating inside a solution factory. Every action is a governed event in a solution lifecycle.

Complexity is managed through hierarchy. The same concepts that govern a personal hobby project also govern a multinational enterprise — the platform scales with the user, never against them.

1.3 Core Entity Hierarchy

```
QALA PLATFORM (Root Solution Factory)
└── Solution Factory
     ├── Solution Factory (nested, child factory)
     └── Solution Development Environment (SDE)
          ├── Solution Model
          │    ├── Solution (instance of a model)
          │    ├── Playbook  (blueprint/design for the solution)
          │    └── Solution Book  (all docs, files, content)
          ├── Toolkit / Toolchain / Toolset
          ├── Repository  (assets, artifacts, capital, resources)
          ├── Content Management System (CMS)
          ├── Communications & Networking Module
          └── Configuration & Settings System
```

Figure 1.1 — Qala Core Entity Hierarchy

1.4 Key Design Principles

Principle	Description
Universal by Design	One platform for every solution type. Personal to enterprise. Software to physical. Digital to regulatory. No assumptions about domain.
Hierarchical Composition	Factories produce factories. Environments compose environments. Solutions nest inside solutions. Hierarchy is unlimited in depth and breadth.
Governance as Foundation	Change control, version management, audit trails, and lifecycle states are not optional features — they are the substrate of every entity.
Distributable & Deployable	SDEs are not locked to one host. They are packaged, versioned, and distributable to any target environment: local, cloud, edge, or offline.
Single Standardized Interface	A solo developer and an enterprise administrator see the same conceptual model, the same workflows, the same vocabulary — at appropriate scale.
Composable Everything	Every SDE, model, playbook, and tool is composable. Any entity can be assembled from reusable components. No lock-in. No silos.
Root Factory Principle	Qala itself is a solution factory. The platform can produce instances of itself. Tiered, hierarchical, self-similar at every level of scale.

2 Solution Factory

A Solution Factory is the primary organizational unit in Qala. It is the container in which Solution Development Environments are created, managed, and governed. Every user, team, or enterprise begins with at least one Solution Factory.

Solution Factory — Organizational Container

A governed namespace that produces and manages Solution Development Environments, child factories, and all solution activity within a defined scope.

Identity: Unique ID, name, slug, description, type (personal / team / enterprise / public), tier
Hierarchy: Parent factory reference (null = root-level). Unlimited child factories and SDEs
Ownership: Owner (user or org), administrators, members, roles, permissions
Policies: Change control policy, release policy, versioning strategy, access controls, audit settings
Templates: Factory-level SDE templates, model templates, playbook templates pre-configured for the factory's domain
Registry: Internal registry of all SDEs, models, solutions, and artifacts produced within the factory
Integrations: Third-party service connections, webhooks, event subscriptions scoped to the factory
Billing & Quota: Resource quotas, usage metering, billing unit (per factory tier)

2.1 Factory Types & Tiers

Factory Type	Intended Use	Key Characteristics	Scale
Personal Factory	Single user managing personal projects, hobbies, experiments.	Single owner, unlimited SDEs, private by default, simplified governance.	Solo
Team Factory	Small group building products or running shared projects.	Multi-member, role-based access, shared templates, team change control.	2–50
Organization Factory	Business managing multiple products, services, or internal tools.	Org-level RBAC, department sub-factories, compliance policies, audit trails.	51–5,000
Enterprise Factory	Large enterprise with complex governance, multi-domain solution estate.	Hierarchical sub-factories, custom domain packs, SSO, data residency, SLA.	5,000+
Platform Factory	A factory that itself produces reusable factory templates and domain packs for others.	Public or gated. Versioned factory blueprints. Marketplace-publishable.	Varies
Root Factory (Qala)	The Qala platform itself. The universal root from which all factories derive.	System-level. Produces all factory types. All domain packs. All platform updates.	Global

2.2 Factory Hierarchy & Nesting

Solution Factories are composable in a tree structure of unlimited depth. A factory at any level can produce child factories, each with its own SDEs, policies, members, and sub-factories. This mirrors how organizations actually operate: divisions within a company, projects within a program, experiments within a lab.

```
Qala Platform  (Root Factory)
└── Acme Corp Factory  (Enterprise Factory)
     ├── Software Products Division  (Org Factory)
     │    ├── Platform Team  (Team Factory)
     │    │    └── api-gateway SDE
     │    └── Consumer Apps Team  (Team Factory)
     │         ├── mobile-app SDE
     │         └── web-app SDE
     ├── Pharma R&D Division  (Org Factory)
     │    └── Formulations Lab  (Team Factory)
     │         ├── formulation-a SDE
     │         └── formulation-b SDE
     └── Financial Products Division  (Org Factory)
          └── Quant Research  (Team Factory)
               └── momentum-strategy SDE
```

Figure 2.1 — Example hierarchical factory structure for a multi-division enterprise

2.3 Factory Governance

Each factory defines its own governance policy, inherited (and optionally overridden) by child factories. Governance configuration includes:

- Change Control Policy — whether CCRs are required, routing rules, approval thresholds by risk level.
- Release Policy — which quality gates must pass before a solution can be released.
- Version Strategy — SemVer, CalVer, custom versioning scheme.
- Audit Policy — what events are logged, retention period, export schedule.
- Access Control — RBAC roles defined at factory level; inheritable by child factories and SDEs.
- Domain Pack — the domain-specific schema extension active in this factory (e.g. Pharma, Financial, Software, Agricultural).

2.4 Factory as a Product

A configured factory — with its templates, domain packs, toolchains, and policies — can itself be packaged as a Factory Blueprint and shared through the Qala Marketplace. Organisations can publish factory blueprints for their industry (e.g. "GxP-Compliant Pharma Factory Blueprint") which other organisations can instantiate and customize. This creates a marketplace of governed, pre-configured factory patterns.

3 Solution Development Environment (SDE)

The Solution Development Environment is the active workspace where solutions are conceived, developed, iterated, tested, and prepared for release. An SDE is not a static folder — it is a living, governed, versioned environment with its own lifecycle.

Solution Development Environment — Active Workspace

A distributable, deployable, composable, version-controlled workspace for developing solution models and building solutions. The primary unit of productive work in Qala.

Identity: Unique ID, name, slug, version, description, type, domain
State: Draft → Active → Suspended → Archived → Deprecated
Versioning: Full version history. Immutable snapshots. Diff between any two versions.
Distribution: Packageable as a distributable artifact. Shareable, cloneable, importable.
Deployment: Deployable to: local, cloud, edge, on-premises, hybrid, or air-gapped targets.
Composability: SDEs can be composed from other SDEs. Sub-environment embedding. Component import.
Configuration: Settings, parameters, options, configuration files. Layered config with inheritance.
Toolchain: Attached toolkits, toolsets, toolchains, and third-party integrations.
Repositories: Built-in asset, artifact, capital, and resource repositories.
CMS: Integrated content management for documentation, media, and structured data.
Communications: Networking modules: messaging, notifications, webhooks, event streams.
Scalability: Horizontally scalable. Resource limits configurable. Auto-scaling policies supported.

3.1 SDE Lifecycle

State	Description	Entry Condition	Exit Condition
Draft	SDE is being configured. Not yet active. Templates and toolchains being assembled.	SDE created	All required configuration complete + activated
Active	SDE is live and in productive use. Solutions can be developed and released.	Activation confirmed	Manual suspend / policy-triggered suspend
Suspended	SDE is paused. All processes halted. Data preserved. Reactivatable.	Manual or policy trigger	Manual reactivation or decomission decision
Archived	SDE work complete. Read-only. All artifacts preserved in long-term storage.	All solutions released or migrated	Cannot be reactivated (clone to new instead)
Deprecated	SDE is end-of-life. Flagged for eventual deletion. Pending data retention policy.	Archive + retention period elapsed	Permanent deletion after retention period

3.2 SDE Configuration System

Every SDE has a layered configuration system. Configuration can be defined at the factory level (inherited by all SDEs), at the SDE level (overrides factory defaults), and at the solution level (overrides SDE defaults).

Config Layer	Scope	Format	Examples
Factory Config	All SDEs in the factory. Inherited by default.	JSON / YAML / TOML	Default version strategy, required quality gates, allowed integrations.
SDE Config	This SDE only. Overrides factory defaults.	JSON / YAML / TOML	Runtime parameters, environment-specific API endpoints, tool versions.
Solution Config	A specific solution within the SDE. Overrides SDE defaults.	JSON / YAML / TOML	Solution-specific feature flags, release targets, notification subscribers.
Runtime Config	Dynamic config injected at runtime (not stored in SDE).	Env vars / Secrets	Credentials, external service URLs, deployment targets.
Config Files	File-based configuration committed to the SDE version history.	Any file format	.qala.yaml (SDE manifest), toolchain.json, domain-pack.json, routes.yaml.

The .qala.yaml SDE Manifest

Every SDE contains a .qala.yaml manifest file at its root. This file defines the SDE identity, domain, version, attached toolchains, repository references, CMS configuration, communication endpoints, and scalability parameters. It is the single source of truth for the SDE's configuration and is version-controlled with the SDE.

```
.qala.yaml  —  SDE Manifest (example)
sde:
  id: sde-a3f9c2                  # Unique SDE identifier
  name: api-gateway-sde           # Human-readable name
  version: 2.4.1                  # Current SDE version
  domain: software                # Domain pack applied
  factory: acme-platform-team     # Parent factory
toolchain:
  - id: node-20-lts               # Pinned runtime
  - id: eslint-8                  # Pinned linter
  - id: qala-ci-pipeline          # CI/CD toolchain
repositories:
  artifacts: repo-artifacts-sde-a3f9c2
  assets:    repo-assets-sde-a3f9c2
scalability:
  min_instances: 1
  max_instances: 10
  auto_scale_policy: cpu_70pct
```

3.3 Toolkits, Toolsets, Toolchains & Tools

An SDE's productive capability comes from its attached tooling. Qala organizes tooling in four levels:

Level	Definition	Example	Relationship
Tool	A single executable capability: a linter, a test runner, a code generator, a data processor.	eslint, pytest, ffmpeg, pandoc	Belongs to a Toolset
Toolset	A coherent collection of tools for a specific task.	Testing toolset: pytest + coverage + mock	Groups Tools. Belongs to a Toolkit
Toolkit	A complete toolbox for a development domain or workflow.	Python Backend Toolkit: language runtime + test + lint + format + type-check	Groups Toolsets. Belongs to a Toolchain
Toolchain	An orchestrated sequence of toolkits producing a governed pipeline: develop → build → test → gate → release.	Qala CI/CD Toolchain for Software SDEs	Orchestrates Toolkits. Attached to an SDE

3.4 Repositories

Each SDE includes four built-in repository types. Repositories are versioned, access-controlled, and queryable through the Qala API:

Repository Type	Contents	Key Capabilities
Artifact Repository	Built outputs: compiled binaries, packages, container images, spec files, test results, signed release bundles.	Push/pull, versioned, integrity-verified (SHA-256), SLSA attestation, retention policy
Asset Repository	Source inputs and design assets: source code, design files, data schemas, configuration templates, raw media.	Version-controlled, diff-capable, linked to SDE version history
Capital Repository	Intellectual and financial capital: IP registrations, patents, licences, research data, proprietary datasets, financial models.	Access-controlled, immutable records, chain-of-custody
Resource Repository	Infrastructure and operational resources: environment definitions, IaC files, capacity reservations, cloud resources, credentials vault (references only).	Scoped secrets, IaC execution context, resource graph

3.5 Content Management System (CMS)

The SDE CMS manages all structured and unstructured content associated with the SDE and its solutions. It bridges the gap between code repositories (where outputs live) and documentation systems (where knowledge lives):

- Structured Content — JSON/YAML data models, schema definitions, domain metadata, reference tables.
- Documentation — technical docs, API references, runbooks, onboarding guides, ADRs (Architecture Decision Records).
- Media — images, diagrams, video demos, design mockups, architecture visualizations.
- Solution Books — the primary documentation container for a given solution (see Section 5.2).
- Playbooks — blueprints, design patterns, and implementation guides (see Section 5.1).
- Version Linking — every content item is linked to the SDE version it was authored under. Historical content is accessible by version.

3.6 Communications & Networking Module

The Communications & Networking Module manages how the SDE connects to the outside world and how its solutions communicate internally and externally:

Capability	Description	Use Cases
Internal Messaging	Async message passing between solutions within the SDE.	Solution-to-solution events, state change notifications, workflow triggers.
External Webhooks	Outbound HTTP callbacks triggered by SDE events.	CI/CD triggers, Slack notifications, external system integrations.
Event Streams	Real-time event bus subscriptions (publish/subscribe).	Live telemetry, audit event feeds, cross-factory event routing.
API Gateway Config	Route definitions, rate limits, authentication requirements for exposed APIs.	Exposing solutions as services to external consumers.
Notification Rules	Policy-based alerting: who gets notified, when, via which channel.	Release approvals, quality gate failures, security events, AI recommendations.
Network Isolation	Configurable ingress/egress rules, VPC peering, private endpoint config.	Air-gapped deployments, regulated environments, zero-trust networking.

3.7 SDE Scalability

SDEs are designed for horizontal scalability from the ground up. Scale is configured per-SDE and per-solution, with policies governing how resources are allocated:

- Compute Scaling — CPU and memory thresholds triggering scale-up/scale-down. Min/max instance bounds per SDE.
- Storage Scaling — Auto-expanding repository storage. Tiered storage (hot/warm/cold) for artifact retention.
- Multi-Region Distribution — SDEs can be distributed across geographic regions for latency optimization or data residency requirements.

- Parallel Execution — Multiple solution builds and test suites run in parallel within an SDE. Queue management for resource-constrained environments.
- Edge Deployment — SDEs can be deployed to edge nodes for low-latency field use (agricultural IoT, point-of-sale, remote lab environments).

4 Solution Model

A Solution Model is the structured definition, schema, and behavioral specification that governs how a specific type of solution is created, validated, and managed. Models are to solutions what blueprints are to buildings — they define the pattern, not the instance.

Solution Model — Definitional Schema

A versioned, reusable definition that specifies the structure, properties, lifecycle states, validation rules, quality gates, and behaviors of a class of solutions. Instantiated to produce Solutions.

Identity: Unique ID, name, version, domain, description, tags
Schema: Field definitions: universal fields + domain-specific fields from Domain Pack
Lifecycle States: Ordered set of valid states and allowed transitions for solutions of this model
Validation Rules: Rules that must pass before a solution can enter each lifecycle state
Quality Gates: Named checkpoints in the lifecycle: what evidence is required, who evaluates
Change Control: Which CCR types apply, what risk levels trigger which approval chains
Release Definition: What constitutes a valid release: required artifacts, required approvals, distribution targets
Composition: Which other models this model can include or extend (model inheritance / composition)
Domain Pack: Reference to the Domain Pack providing domain-specific extensions to the universal model

4.1 The Universal Solution Schema

Every solution model shares a universal core schema. Domain packs add fields on top of this core without replacing it:

Field Group	Fields	Always Present
Identity	id, name, slug, version, type, domain, tags, labels	Yes
Ownership	owner_id, created_by, last_modified_by, assigned_to	Yes
Lifecycle	state, state_history, previous_state, transition_reason, transition_timestamp	Yes
Versioning	version, version_history, version_notes, changelog, is_latest, is_stable, is_deprecated	Yes
Associations	parent_solution_id, child_solution_ids, related_solution_ids, sde_id, factory_id, model_id	Yes
Quality	quality_gate_results, open_defects, defect_summary, last_gate_evaluation	Yes
Change Control	open_ccr_count, ccr_ids, last_ccr_id, change_freeze_until	Yes
Artefacts	artefact_ids, primary_artefact_id, artefact_registry_ref	Yes
Documentation	playbook_id, solution_book_id, readme_url, doc_version	Yes
Compliance	compliance_status, regulatory_declarations, evidence_package_ids, last_audit_date	Yes

AI	ai_recommendations, last_ai_analysis, risk_score, drift_status	Yes
Domain Fields	All fields defined by the active Domain Pack for this solution type	Domain-specific

4.2 Domain Packs

A Domain Pack is a versioned JSON schema extension that adds domain-specific fields, validation rules, lifecycle states, and quality gate definitions to the Universal Solution Schema. Domain Packs are the mechanism by which Qala adapts to any industry or workflow without changing the platform core.

Domain Pack	Adds to Universal Schema	Example Fields	Quality Gate Types
Software	Runtime, language, dependencies, test coverage, SAST results.	language, runtime_version, test_coverage_pct, sast_passed	SAST, Coverage, Dependency Check, SLSA Attestation
CPG / Pharma	INCI names, batch info, regulatory filings, stability data.	inci_name, batch_type, eu_reach_status, stability_months	Batch Release Gate, QP Sign-off, Regulatory Pre-check
Financial	Strategy parameters, risk metrics, regulatory filings, mandates.	strategy_type, sharpe_ratio, var_99, mifid_algorithm_registered	Risk Gate, Drawdown Gate, Mandate Compliance Gate
Agricultural	Crop data, field coordinates, treatment protocols, yield targets.	crop_type, field_gps, treatment_protocol_id, yield_target_tha	Agronomic Gate, Traceability Gate, Seasonal Gate
Legal / Tax	Jurisdiction, effective dates, legislative reference, filing status.	jurisdiction_code, effective_date, legislative_ref, filing_status	Legal Review Gate, Compliance Gate, Effective-Date Gate
Research	Hypothesis, experimental design, dataset versions, IRB approval.	hypothesis, experiment_design, dataset_version, irb_approved	Peer Review Gate, Reproducibility Gate, IRB Gate

4.3 Model Composition & Inheritance

Solution models can extend other models through composition and inheritance, enabling a rich library of reusable model patterns:

- Base Model — a minimal model defining universal behavior. All other models inherit from it.
- Domain Model — a base model extended with a Domain Pack. E.g. "Software Solution Model" = Base + Software Domain Pack.
- Specialization Model — a domain model further specialized. E.g. "Microservice Solution Model" extends "Software Solution Model" with microservice-specific fields.
- Composite Model — a model that includes other models as components. E.g. "Platform Solution Model" composes a Frontend Model, Backend Model, and Infrastructure Model.
- Custom Model — an organization-defined model extending any of the above with proprietary fields and rules.

5 Solutions, Playbooks & Solution Books

5.1 Solution

A Solution is an instance of a Solution Model. It is the actual artifact being governed — the software application, the pharmaceutical formulation, the financial model, the project plan, the managed service, the personal automation. Solutions live inside SDEs, follow their model's lifecycle, and accumulate versions, changes, releases, and artifacts over their lifetime.

Solution — Governed Instance

A concrete, versioned instance of a Solution Model. The actual thing being built, managed, and delivered. Has its own lifecycle, change history, release record, and associated playbook and solution book.

Identity: Follows Universal Solution Schema (Section 4.1). Typed by its Solution Model.
Lifecycle: Progresses through model-defined lifecycle states. Each transition is a governed event.
Versions: Every meaningful change creates a new version. Immutable version history. Rollback-capable.
Changes: Every intentional modification goes through a Change Control Request (CCR).
Releases: Solutions are released through the SDE's Release Engine. Each release is a governed, versioned, attested event.
Playbook: An associated Playbook defines the design, blueprint, and implementation guide for the solution.
Solution Book: An associated Solution Book contains all documentation, files, and content for the solution.
Artefacts: Artifacts produced by the solution (builds, specifications, test reports) are stored in the SDE's Artifact Repository.
AI: The Qala AI Agent monitors the solution continuously: health, drift, risk, compliance posture.

5.2 Solution Lifecycle States

The specific lifecycle states for a solution depend on its model. The universal base model provides a common state progression. Domain packs can add or rename states. The following is the universal base lifecycle:

State	Description	Key Allowed Actions
Conception	Idea captured. Not yet in development. Goal, purpose, and scope defined.	Define scope, attach model, assign SDE, create playbook outline
Draft	Active development. Solution being designed, built, and iterated.	Edit, commit, test, iterate, create CCRs for design changes
In Review	Development complete. Undergoing evaluation: quality checks, peer review, approval.	Submit for review, evaluate quality gates, leave review feedback

Approved	Review passed. Solution authorized for release.	Create release, generate artifacts, prepare release package
Released	Solution is packaged, attested, and available for deployment or distribution.	Deploy, distribute, install, rollback to previous release
Live / Active	Solution is deployed and operational in its target environment.	Monitor, observe, receive CCRs for operational changes
Deprecated	Solution is end-of-life. Still operational but flagged for retirement.	Notify consumers, create migration CCR, set retirement date
Retired	Solution is no longer operational. Fully decommissioned. Historical record preserved.	Read-only access to history. Archive artifacts.

5.3 Playbook

A Playbook is the blueprint and design document for a solution. It defines how the solution should be built, what patterns to follow, what decisions have been made, and why. Playbooks are living documents — they evolve with the solution and are version-controlled alongside it.

Playbook — Blueprint & Design Document

The design specification, architectural blueprint, and decision record for a solution. Defines how the solution is built, not what it does — that is the Solution Book's job.

Purpose: Captures design decisions, architectural choices, implementation patterns, and rationale.

Structure: Modular: Problem Statement, Goals, Design Decisions (ADRs), Architecture Diagram, Component Map, Integration Map, Quality Strategy, Deployment Plan, Rollback Plan.

Versioning: Every playbook is versioned. Major design changes create a new playbook version linked to the solution version.

Templates: Playbook templates provided per model type. Software, CPG, Financial, and other domain playbook templates available out-of-box.

Composability: Playbooks can include sections from other playbooks (reusable design patterns).

AI Drafting: The AI Agent can auto-draft playbook sections from the solution's context, history, and domain knowledge.

5.4 Solution Book

A Solution Book is the comprehensive documentation and content repository for a solution. Whereas the Playbook is the design specification (how to build it), the Solution Book is the operational encyclopedia (everything about it). It is generated and maintained throughout the solution's lifecycle.

Solution Book — Complete Knowledge Repository

All documentation, files, structured data, media, and content for a given solution — from inception through retirement. The single source of truth for everything about the solution.

Contents: Sections: Overview, Goals & Objectives, Stakeholders, Functional Specs, Technical Specs, Data Models, API Reference, Integrations, Runbooks, Incident History, Change Log, Release Notes, Compliance Evidence, Retirement Record.

File Formats: Markdown, PDF, Word, HTML, JSON, YAML, CSV, images, diagrams (Mermaid, Draw.io), video, and any binary file type.

Auto-Generated: Many sections auto-populate from platform data: change log from CCR history, release notes from release records, compliance evidence from audit events.

AI Assist: AI Agent can generate, summarize, and update Solution Book sections from platform activity.

Versioning: Every section of the Solution Book is versioned and linked to the solution version it describes. Historical snapshots preserved.

Accessibility: Accessible to: solution owner, SDE members, factory admins, read-only guests (configurable). Exportable as a PDF package.

Publishing: Solution Books can be published to the Qala Marketplace (for public solutions) or internal knowledge portals.

5.5 Solution Relationships & Composition

Solutions can relate to and compose from other solutions. This models real-world dependency and composition patterns:

Relationship Type	Description	Example
Parent → Child	A solution contains sub-solutions. Parent lifecycle transitions can cascade to children.	Platform Solution contains: Frontend, Backend, Infrastructure, and DataStore sub-solutions.
Dependency	Solution A requires Solution B. Upgrade of B triggers change control in A.	Mobile App depends on api-gateway. api-gateway update requires mobile-app CCR review.
Composition	Solution is assembled from multiple component solutions.	"Order Management System" composed from: Search, Cart, Checkout, Fulfilment sub-solutions.
Derivation	Solution B is derived from Solution A (fork). Independent lifecycle but shared history.	enterprise-edition forked from community-edition. Independent versioning, shared origin.
Cross-Factory	Solution in Factory A produces artifacts consumed by Solution in Factory B.	Shared Design System in Design Factory consumed by 14 product SDEs across 3 product factories.

6 Change Control & Versioning

Every intentional change to any solution, SDE, model, or factory in Qala is a governed event. Change Control Requests (CCRs) are the mechanism through which changes are proposed, evaluated, approved, and executed. Versioning ensures that the complete history of every entity is preserved and auditable.

6.1 Change Control Request (CCR)

Change Control Request (CCR) — Governed Change Record

A structured, auditable record of a proposed change to a solution, SDE, model, or factory. The primary governance instrument for all intentional modifications.

Identity: CCR ID, title, type, status, priority, risk score
Scope: Affected solutions, SDEs, and factory refs. Impact analysis (cascading effects).
Content: Change description, justification, implementation plan, rollback plan, regulatory implications.
Risk: Computed risk score (1–10) based on: change type, impact scope, regulatory dimension, domain risk factors.
Routing: Approval chain auto-computed from: risk level, solution type, factory CCR policy, domain compliance rules.
Evidence: Supporting artifacts attached: test results, simulation outputs, regulatory pre-checks, expert reviews.
Status: Draft → Submitted → In Review → Approved / Rejected / Deferred → Implemented → Verified
Audit: Every status change recorded with actor, timestamp, and reason. Immutable audit trail.

6.2 CCR Types

CCR Type	When Used	Auto-Applies To
Feature Addition	Adding new capability to a solution.	All solution types.
Defect Fix	Correcting an error, bug, or non-conformance.	All solution types.
Configuration Change	Modifying settings, parameters, or environment configuration.	All solution types.
Dependency Update	Updating a referenced tool, library, integration, or service.	Software, Platform, Research solutions.
Specification Update	Updating the design specification or requirements documentation.	All solution types.
Ingredient Substitution	Replacing a component material or input substance.	CPG, Pharma, Agricultural solutions.
Mandate Amendment	Modifying the governing mandate, policy, or scope of a solution.	Financial, Legal, Service, Tax solutions.

Legislative Update	Updating a solution to reflect a change in law or regulation.	Tax, Legal, Compliance, Financial solutions.
Security Patch	Addressing a security vulnerability or threat.	Software, Platform, Toolchain solutions.
Emergency Change	An urgent unplanned change bypassing standard routing. Requires post-facto review.	All solution types. Elevated audit trail.
Model Update	Changing the Solution Model itself (schema, lifecycle, gates).	Requires factory-admin approval or above.

6.3 Risk Scoring

CCR risk scores are computed automatically from a weighted scoring model. Scores determine the approval chain required:

Risk Band	Score Range	Approval Required	Example
Low	1–3	Solution owner or auto-approve if policy allows.	Updating a README. Bumping a minor dependency. Adding a config option.
Medium	4–6	Two-stage: Technical lead + domain expert.	Feature addition affecting 5–10 solutions. New third-party integration.
High	7–8	Three-stage: Tech lead + domain expert + compliance.	Change to regulatory-relevant field. Cross-domain impact. Ingredient substitution.
Critical	9–10	Full approval board (CCB). May require external sign-off.	Recall initiation. Emergency change affecting 50+ solutions. Structural model change.

6.4 Versioning

Versioning in Qala applies to every entity: solutions, SDEs, solution models, factories, playbooks, solution books, toolchains, and domain packs. The platform supports three versioning strategies:

- Semantic Versioning (SemVer) — MAJOR.MINOR.PATCH (e.g. 2.4.1). Default for software, platform, and library solutions.
- Calendar Versioning (CalVer) — YYYY.MM.DD or YYYY.MM (e.g. 2025.03). Common for tax rules, regulatory solutions, and compliance frameworks.
- Sequential Versioning — incrementing integers (e.g. v42). Used for batch records, experiment runs, and iteration-heavy research solutions.

Immutable Version Principle

Once a version is released, it is immutable. It cannot be deleted, edited, or replaced. A new change creates a new version. This ensures that every released artifact has a permanent, auditable record. Rollback means activating a previous immutable version — never modifying history.

7 Users, Roles & Personas

Qala is designed for users across the full spectrum: a solo developer organizing a side project and a Chief Compliance Officer governing 10,000 solutions at a global enterprise both use the same platform through the same conceptual model, at appropriate scale.

7.1 Platform Roles

Role	Scope	Key Permissions
Root Admin	Qala platform	Full system access. Manage all tenants, factories, and platform settings. Available only to Qala operators.
Factory Admin	One factory + children	Create/manage SDEs, define policies, invite members, manage domain packs, view all audit trails.
Factory Member	One factory	Access to SDEs they are assigned to. Create solutions, submit CCRs, create releases (per role).
SDE Owner	One SDE	Full control of their SDE: configuration, toolchains, members, repositories.
Solution Developer	One or more SDEs	Create and modify solutions, commit changes, run builds, submit CCRs.
Reviewer	One or more SDEs	Review and approve/reject CCRs and releases assigned to them. Cannot create solutions.
Release Manager	One or more factories	Approve and trigger releases. Manage quality gate evaluations.
Compliance Officer	One or more factories	View all compliance evidence, generate audit packages, access full audit trail. Read-only on solutions.
Guest / Reader	Specified scope	Read-only access to specified solutions, SDEs, or factories. Cannot create or modify.
AI Agent	System-level	Read all governed data. Write AI recommendations and analysis results. Cannot execute state transitions.

7.2 User Scale Profiles

Qala's design explicitly supports the full range of user scale without changing the platform's core model. The same entities exist at every scale — the complexity and governance depth simply adjust:

Profile	Context	Typical Setup	Governance Depth
Solo Hobbyist	One person managing personal projects, experiments, homelab, automation.	1 Personal Factory, 3–10 SDEs, simplified CCR (optional), no approval routing.	Minimal

Freelancer / Indie Dev	Individual professional managing client projects and their own products.	1–2 Personal Factories, 5–20 SDEs, lightweight CCR, no-approval for low-risk.	Low
Small Team Startup	Founding team building a product.	1 Team Factory, 10–30 SDEs, CCR with peer review, 2-stage approvals.	Moderate
Growing SMB	Company with multiple products, services, and internal tools.	1–3 Org Factories, 30–200 SDEs, CCR with domain routing, compliance policies.	High
Enterprise Division	A business unit of a large enterprise managing a complex solution estate.	3–10 Org Factories (nested), 200–2000 SDEs, full CCR routing, audit trails, domain packs.	Very High
Global Enterprise	Multi-national organization with complex governance, regulatory, and compliance needs.	10–50 Org Factories, 2000+ SDEs, CCB approval chains, data residency, SOC 2, GxP.	Maximum

7.3 Domain Personas

The Qala UI adapts its vocabulary and navigation to the user's primary domain. Personas are configured at login and adjustable in Profile Settings. Below are the primary persona types:

Persona	Primary Domain	Vocabulary Adaptation (SDE label)	Primary Actions
Software Developer	Software / Platform	"SDE" → "SDE"	Commit, build, test, release, deploy
Formulation Chemist	CPG / Pharma	"SDE" → "Formulation Workbench"	Formulate, simulate, test, batch release
Portfolio Manager	Financial	"SDE" → "Research Workbench"	Backtest, risk-gate, mandate, activate
Farm Operations Manager	Agricultural	"SDE" → "Field Environment"	Monitor, irrigate, harvest, trace
Principal Consultant	Professional Services	"SDE" → "Knowledge Workspace"	Develop methodology, engage, deliver, publish
Research Scientist	Research / Academia	"SDE" → "Research Environment"	Experiment, analyse, peer-review, publish
Tax Product Manager	Legal / Tax	"SDE" → "Tax Workspace"	Draft rule set, validate, schedule, activate
Compliance Officer	Cross-domain	"SDE" → "Compliance Environment"	Audit, evidence, report, certificate

8 Platform Architecture

Qala is architected as a multi-tenant, cloud-native platform with a strict layered design. Each layer is independent, testable, and replaceable. The platform is designed to be run as a SaaS service, as a self-hosted enterprise installation, or as an embedded component within another platform.

8.1 Architectural Layers

```
┌──────────────────────────────────────────────────────────────────────────┐
│ PRESENTATION LAYER                                                       │
│ Web App (React)  ·  Mobile App (React Native)  ·  CLI  ·  VS Code Extension │
├──────────────────────────────────────────────────────────────────────────┤
│ API GATEWAY LAYER                                                        │
│ REST API  ·  GraphQL API  ·  WebSocket API  ·  Webhook Engine            │
├──────────────────────────────────────────────────────────────────────────┤
│ DOMAIN SERVICES LAYER                                                    │
│ Factory Service  ·  SDE Service  ·  Solution Service  ·  CCR Service     │
│ Release Engine  ·  Audit Service  ·  CMS Service  ·  Repository Service  │
├──────────────────────────────────────────────────────────────────────────┤
│ INTELLIGENCE LAYER                                                       │
│ AI Agent  ·  Recommendation Engine  ·  Drift Monitor  ·  Compliance AI   │
├──────────────────────────────────────────────────────────────────────────┤
│ DATA LAYER                                                               │
│ Solution DB  ·  Event Store  ·  Audit Vault  ·  Search Index             │
│ Object Storage  ·  Repository Store  ·  Secrets Vault                    │
└──────────────────────────────────────────────────────────────────────────┘
```

Figure 8.1 — Qala Platform Layered Architecture

8.2 The Root Factory Principle

Qala is itself a Solution Factory — the Root Solution Factory. This is not a metaphor; it is an architectural reality. The Qala platform is managed through itself: new domain packs are developed in Qala SDEs, platform features are solution models within the Qala factory, and the platform's own release pipeline runs on the same Release Engine that governs all other solutions.

> **Self-Describing Architecture**
>
> *Qala's own development, versioning, and release is governed by Qala. Every platform update goes through a CCR. Every release is a governed release event. Every architectural decision is recorded as an ADR in the Qala root factory's Playbooks.*
>
> *This makes Qala a self-describing, self-governing system — the platform is a living demonstration of its own capabilities.*

8.3 Deployment Models

Model	Description	Best For

Qala SaaS (Multi-tenant)	Managed by Qala. Shared infrastructure. Per-factory isolation via encryption and namespace separation.	Individuals, SMBs, most enterprises.
Qala SaaS (Single-tenant)	Managed by Qala. Dedicated infrastructure for one customer. Full isolation.	Enterprises with strict data isolation requirements.
Self-Hosted (Cloud)	Customer installs Qala on their own cloud (AWS, Azure, GCP). Qala provides a Helm chart and operator.	Enterprises requiring data sovereignty or custom cloud configuration.
Self-Hosted (On-Premises)	Customer installs Qala on their own data centre infrastructure.	Regulated industries with on-premises mandates (defence, government, healthcare).
Air-Gapped	Fully isolated installation with no external network access. Manual update packages.	Classified environments, critical infrastructure, maximum security.
Edge / Embedded	Lightweight Qala node deployed to edge devices (IoT gateways, field equipment).	Agricultural IoT, point-of-sale, remote lab or field operations.

8.4 Integration Architecture

Qala integrates with the existing tool landscape through a standardized integration framework. Integrations are first-class citizens in the SDE toolchain system:

- Native Integrations — maintained by Qala for the most common platforms: GitHub, GitLab, Jira, Confluence, Slack, Microsoft Teams, SAP, Salesforce, ServiceNow, AWS, Azure, GCP.
- Domain Integrations — industry-specific: LIMS (pharma/CPG), Bloomberg (financial), EPCIS (supply chain), FAOSTAT (agricultural), EDGAR (financial regulatory).
- API-First — all Qala capabilities are available via REST and GraphQL APIs. Any tool can integrate with Qala through standard HTTP.
- Webhook Engine — any Qala event can trigger an outbound webhook to any external system.
- Import/Export — solutions, SDEs, and factories can be exported as structured packages and imported into any Qala instance.

9 Complete Entity Reference

A consolidated reference of all platform entities, their identity, and their relationships.

9.1 Entity Summary Table

Entity	Layer	Produced By	Produces	Governed By
Qala Platform	Root	N/A (the root)	All entity types	Qala platform governanc
Solution Factory	Organizational	Qala Platform or Parent Factory	Child Factories, SDEs	Factory policies, RBAC
Solution Dev Env (SDE)	Workspace	Solution Factory	Solutions, Artifacts, Content	SDE config, factory polic
Solution Model	Schema/Blueprint	Factory Admin / Qala Domain Team	Solutions (instances)	Model versioning, doma pack
Domain Pack	Schema Extension	Qala or custom model authors	Domain-specific field extensions	Pack versioning, factory approval
Solution	Output	SDE (via developer action)	Artifacts, Releases, change history	CCR, lifecycle state mac model
Playbook	Documentation	SDE member or AI Agent	Design decisions, ADRs, blueprints	Solution versioning, CM
Solution Book	Documentation	Auto-generated + SDE members	Comprehensive solution documentation	Solution versioning, CM access policy
CCR	Governance	Any authorized user	Approved changes, compliance evidence	Risk routing, approval ch policy
Release	Deployment	Release Manager	Deployed solution, release artifacts	Release policy, quality g
Artifact	Output	SDE build/toolchain	Deployable packages, evidence records	Artifact repository policie
Toolchain	Infrastructure	Qala Marketplace or custom	Built artifacts, test results	Toolchain versioning, SD config
AI Agent	Intelligence	Qala platform (system)	Recommendations, risk scores, insights	AI policy, privacy setting
Audit Event	Compliance	Every governed action (automatic)	Compliance evidence packages	Audit vault (immutable)

9.2 Qala Platform — Full Hierarchy

```
QALA PLATFORM  (Root Solution Factory)
│
├── SOLUTION FACTORIES (one or many per tenant)
│     ├── Child Solution Factories (unlimited nesting)
│     └── SOLUTION DEVELOPMENT ENVIRONMENTS (SDEs)
```

```
│          ├── Configuration System
│          │    ├── SDE Manifest (.qala.yaml)
│          │    ├── Settings + Parameters + Options
│          │    └── Configuration Files (layered: Factory > SDE > Solution)
│          ├── Toolchain
│          │    ├── Toolkits
│          │    │    └── Toolsets  →  Tools
│          │    └── Third-party Integrations
│          ├── Repositories
│          │    ├── Artifact Repository
│          │    ├── Asset Repository
│          │    ├── Capital Repository
│          │    └── Resource Repository
│          ├── Content Management System (CMS)
│          │    ├── Playbooks
│          │    └── Solution Books
│          ├── Communications & Networking Module
│          └── SOLUTION MODELS
│               └── SOLUTIONS (instances of models)
│                    ├── Playbook (blueprint)
│                    ├── Solution Book (all docs + content)
│                    ├── Artifacts (in Artifact Repository)
│                    ├── Change Control Requests (CCRs)
│                    └── Releases
│
├── PLATFORM SERVICES (cross-cutting)
│    ├── AI Agent
│    ├── Audit Vault
│    ├── Identity & RBAC
│    ├── Event Bus
│    └── Compliance Evidence Engine
│
└── MARKETPLACE
     ├── Domain Packs
     ├── Factory Blueprints
     ├── Playbook Templates
     └── Toolchains
```

Figure 9.1 — Complete Qala Platform Entity Hierarchy

End of Document — Qala Platform Concept & Architecture v1.0